

The AI/ML Model Assembly Line: A Modular Step-by-Step Guide

Think of building an AI/ML model like cooking a meal — you have ingredients (data), a recipe (algorithm), cooking steps (pipeline), and you taste-test along the way (testing). The beauty is that you can rearrange the steps to cook different dishes (different algorithms) using the same kitchen setup.

The 10 Core Steps (Like Building Blocks)

Step	What It Does	Plain English
1. SCOPE	Define the problem and success metrics.	"What are we trying to predict? How will we know if we're right?"
2. ACQUIRE	Gather raw data from databases, APIs, or sensors.	"Get all the ingredients from different places."
3. EXPLORE (EDA)	Find patterns, missing pieces, and outliers.	"Check what's fresh, what's spoiled, and what's missing."
4. PREPARE	Clean, fix, and transform data into usable formats.	"Wash vegetables, chop them, and measure portions."
5. ENGINEER	Create meaningful signals from raw data.	"Turn 'flour + eggs' into 'batter'—the right mix for baking."
6. SPLIT DATA	Divide into training, validation, and test sets.	"Set aside a sample to taste-test later."

Step	What It Does	Plain English
7. TRAIN	Feed data to an algorithm so it learns patterns.	"Cook the dish—let the flavors blend and learn."
8. VALIDATE	Check performance on unseen data.	"Taste-test with the sample you set aside."
9. DEPLOY	Put the model into production for predictions.	"Serve the dish to customers."
10. MONITOR	Watch performance over time and update.	"Check if customers still like it. Update the recipe if needed."

How These Steps Flow Together

The Standard Path

SCOPE → ACQUIRE → EXPLORE → PREPARE → ENGINEER → SPLIT → TRAIN → VALIDATE → DEPLOY → MONITOR

The Real-World Iterative Path

In real projects, the process is not a straight line. You go back and forth between stages as you learn more about your data and model performance.

SCOPE ↔ ACQUIRE ↔ EXPLORE ↔ PREPARE ↔ ENGINEER ↔ SPLIT ↔ TRAIN ↔ VALIDATE ↔ DEPLOY ↔ MONITOR

How to Rearrange Steps for Different Algorithms

Each step is a modular building block. You can reuse, skip, or reorder them to build different types of models:

- **Example 1: Image Recognition (Deep Learning)**

ACQUIRE (images) → EXPLORE → PREPARE (resize) → ENGINEER (extract features) → SPLIT → TRAIN (neural net) → VALIDATE → DEPLOY.

- **Example 2: Real-Time Fraud Detection**

ACQUIRE (streaming data) → PREPARE (real-time cleaning) → ENGINEER (fraud signals) → TRAIN (anomaly detection) → DEPLOY (as API) → MONITOR → RETRAIN (daily).

- **Example 3: Simple Rule-Based System (No ML)**

SCOPE → ACQUIRE → EXPLORE → (skip TRAIN) → write RULES manually → VALIDATE → DEPLOY.

- **Example 4: Transfer Learning**

SCOPE → ACQUIRE → EXPLORE → PREPARE → (skip full TRAIN) → fine-tune PRE-TRAINED model → VALIDATE → DEPLOY.

The "Orchestration" Concept

To make this work smoothly, you connect steps using orchestration—like a conductor directing an orchestra:

1. **Output to Input:** Each step produces an output (e.g., cleaned data) that the next step uses as input.
2. **Execution:** Steps can run in sequence or in parallel.
3. **Resilience:** If a step fails, the system can retry or stop gracefully.

Quick Reference: Step Dependencies

To do this step...	You need these steps done first
<u>PREPARE</u>	ACQUIRE
ENGINEER FEATURES	PREPARE
TRAIN	ENGINEER + SPLIT
VALIDATE	TRAIN

To do this step...	You need these steps done first
DEPLOY	VALIDATE
RETRAIN	MONITOR (feeds back to TRAIN)

Summary: Think of It Like LEGO Bricks

- **10 Core Steps** = Your LEGO bricks.
- **Standard Flow** = One way to assemble them.
- **Reordering/Skipping** = Building different things with the same bricks.
- **Orchestration** = The instructions that tell which brick connects to which.
- **Iteration** = Building, taking apart, and building again better.

Building Your Personal n8n-Style AI/ML Workflow App

As a CS&IT industrialist working with regression and anomaly detection, you need a modular, visual workflow orchestrator that lets you chain data science steps like LEGO bricks. Here's a complete blueprint to build it.

1. Core Philosophy: Think "Workflow as LEGO"

n8n's architecture follows a layered + plugin-based approach. The key insight: each ML stage is a node that:

- Accepts inputs (data, parameters)
- Produces outputs (transformed data, models, visualizations)
- Can be rearranged to create different algorithms

None

```
workflow (abstract model) ← core (execution engine) ← cli (API layer) ← editor-ui (visual canvas)
```

Think of it as:

- **workflow** = the blueprint of your ML pipeline
- **core** = the engine that runs it step by step

- **cli** = REST APIs, auth, persistence
- **editor-ui** = drag-drop canvas where users build pipelines

2. Technology Stack (Python-First)

Layer	Technology	Why
Frontend	React + ReactFlow + TypeScript	Visual drag-drop canvas
Backend	FastAPI (Python)	High-performance, auto-docs, async
Database	PostgreSQL + SQLAlchemy	Workflow persistence, user data
Task Queue	Celery + Redis	Async execution, retries, scheduling
ML Engine	Python (pandas, scikit-learn, tensorflow)	Your regression/anomaly models
Storage	S3 (boto3)	Dataset storage, model artifacts
Auth	JWT	Secure user access

3. Modular Page Design

Page 1: SCOPE — Problem Definition

Purpose: User defines what problem they're solving and what success looks like.

UI Components:

- Text input: "Project Name"
- Dropdown: "Problem Type" (Regression / Anomaly Detection / Classification)
- Text area: "Business Problem Description"
- Text input: "Success Metric" (e.g., RMSE < 0.05, Precision > 0.90)
- File upload: "Upload requirements document (optional)"

Backend Logic:

Python

```
# Store scope as workflow metadata
```

```
workflow = {  
    "id": "wf_123",  
    "name": "Sales Forecast",  
    "problem_type": "regression",  
    "success_metric": "RMSE < 0.05",  
    "status": "scoped"  
}
```

Output: Workflow ID + metadata stored in database.

Page 2: ACQUIRE — Dataset Selection from S3

Purpose: User selects a dataset from S3 buckets.

UI Components:

- List of S3 buckets (pre-configured)
 - File browser showing CSV/Parquet/JSON files
- Dataset preview (first 5 rows)
 - Schema detection display
- "Select Dataset" button

Backend Logic:

Python

```
import boto3  
s3 = boto3.client('s3')  
# List buckets  
buckets = s3.list_buckets()  
# List files in selected bucket  
objects = s3.list_objects_v2(Bucket=bucket_name)  
# Preview dataset  
df = pd.read_csv(s3_url)  
preview = df.head().to_dict()  
schema = df.dtypes.to_dict()
```

Output: dataset_path (S3 URI) + schema + preview stored in workflow.

Page 3: EXPLORE (EDA) — Visual Analysis with Highlights

Purpose: User explores data, finds patterns, missing values, and weird data points.

Critical: Mark and highlight issues.

UI Components:

- Summary Statistics Table: count, mean, std, min, max for each column
- Missing Values Heatmap: visual grid showing missing data
- Distribution Charts: histograms for numerical, bar charts for categorical
- Correlation Matrix: heatmap showing feature relationships
- Anomaly Highlighting: points flagged as outliers (marked in red)
- Text Analysis Report: auto-generated natural language summary

Backend Logic:

Python

```
import pandas as pd
import numpy as np
from scipy import stats
df = pd.read_csv(dataset_path)

# 1. Summary statistics
summary = df.describe(include='all').to_dict()

# 2. Missing values
missing = df.isnull().sum().to_dict()
missing_pct = (df.isnull().sum() / len(df) * 100).to_dict()

# 3. Outlier detection (Z-score method for anomaly detection)
z_scores =
np.abs(stats.zscore(df.select_dtypes(include=[np.number])))
outliers = (z_scores > 3).sum().to_dict()

# 4. Generate text report
report = f"""
**Data Overview:**
- {len(df)} rows, {len(df.columns)} columns
- Missing values found in {sum(1 for v in missing.values() if v > 0)} columns
- {sum(outliers.values())} potential outliers detected
**Key Insights:**
- {df.columns[0]} has the highest missing rate at {max(missing_pct.values()):.1f}%
```

```
- Strong correlation detected between {correlated_pairs}
- Distribution of {target_column} appears {distribution_type}
"""
```

Output: eda_report (text) + visualizations (chart images/base64) + outlier_indices + missing_columns stored.

Page 4: PREPARE — Data Cleaning & Transformation

Purpose: User cleans, fixes, and transforms data into usable format.

UI Components:

- Missing Value Handler: Drop rows / Drop columns / Fill with mean/median/mode / Forward fill
- Outlier Handler: Cap / Remove / Transform (log, sqrt)
- Data Type Converter: Convert columns to appropriate types (int, float, category, datetime)
- Column Operations: Rename columns / Drop columns / Reorder columns
- Preview: Show before/after comparison

Backend Logic:

Python

```
def prepare_data(df, operations):
    for op in operations:
        if op['type'] == 'fill_missing':
            df[op['column']] =
df[op['column']].fillna(op['value'])
        elif op['type'] == 'drop_rows':
            df = df.dropna()
        elif op['type'] == 'remove_outliers':
            # Using IQR method
            Q1 = df[op['column']].quantile(0.25)
            Q3 = df[op['column']].quantile(0.75)
            IQR = Q3 - Q1
            df = df[(df[op['column']] >= Q1 - 1.5*IQR)
& (df[op['column']] <= Q3 + 1.5*IQR)]
        elif op['type'] == 'log_transform':
```



```

        df[op['column']] =
np.log1p(df[op['column']])
    return df

```

Output: cleaned_dataset_path (S3 URI) + applied_operations list.

Page 5: ENGINEER FEATURES — Feature Creation

Purpose: Create new meaningful signals from raw data.

UI Components:

- Feature Types: Polynomial, Interaction, Aggregation, Date features, Text features
- Feature Selection: Checkboxes to select features, Correlation-based recommendation
- Preview: Show new feature set

Backend Logic:

Python

```

from sklearn.preprocessing import PolynomialFeatures
def engineer_features(df, operations):
    for op in operations:
        if op['type'] == 'polynomial':
            poly =
PolynomialFeatures(degree=op['degree'],
include_bias=False)
            poly_features =
poly.fit_transform(df[op['columns']])
            # Add to dataframe with names
        elif op['type'] == 'interaction':
            df[f"{op['col1']}_{op['col2']}"] =
df[op['col1']] * df[op['col2']]
        elif op['type'] == 'date_features':
            df['day'] = df[op['column']].dt.day
            df['month'] =
df[op['column']].dt.month
            df['year'] =
df[op['column']].dt.year

```

```
return df
```

Output: featured_dataset_path + feature_names list.

Page 6: SPLIT DATA — Train/Validation/Test Split

Purpose: Divide data into training, validation, and test sets.

UI Components:

- Split Ratios: Sliders (e.g., 70/15/15)
- Stratification: Toggle for stratified split
- Random Seed: Input for reproducibility
- Preview: Show sizes of each split

Backend Logic:

Python

```
from sklearn.model_selection import train_test_split
X = df.drop(columns=[target_column])
y = df[target_column]
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=(val_ratio + test_ratio), random_state=seed
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=test_ratio/(val_ratio+test_ratio),
    random_state=seed
)
```

Output: split_indices dict + paths to train/val/test datasets in S3.

Page 7: TRAIN — Model Training

Purpose: Feed data to algorithm so it learns patterns.

UI Components:

- Algorithm Selection: Regression / Anomaly Detection algorithms
- Hyperparameters: Dynamic form based on algorithm
- Training Logs: Real-time output streaming
- Progress Bar: Training progress indicator

Backend Logic:

Python

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import IsolationForest
if problem_type == 'regression':
    model = RandomForestRegressor(
        n_estimators=params['n_estimators'],
        max_depth=params['max_depth'],
        random_state=42
    )
    model.fit(X_train, y_train)
elif problem_type == 'anomaly':
    model = IsolationForest(
        contamination=params['contamination'],
        random_state=42
    )
    model.fit(X_train) # Unsupervised
```

Output: model_artifact (pickle/onnx) saved to S3 + training_metrics.

Page 8: VALIDATE & TEST — Model Evaluation

Purpose: Check if model works correctly on unseen data.

UI Components:

- Metrics Dashboard: Regression / Anomaly metrics
- Visualizations: Scatter plots, Residuals, Confusion matrix
- Feature Importance: Bar chart
- Threshold Tuner: Adjust decision threshold

Backend Logic:

Python

```
from sklearn.metrics import mean_squared_error, r2_score
y_pred = model.predict(X_test)
metrics = {
    'rmse': np.sqrt(mean_squared_error(y_test, y_pred)),
    'r2': r2_score(y_test, y_pred),
    'mae': mean_absolute_error(y_test, y_pred),
    'predictions': y_pred.tolist()
```

```

}
# Generate validation report
validation_report = f"""
**Model Performance on Test Set:**
- RMSE: {metrics['rmse']:.4f}
- R2 Score: {metrics['r2']:.4f}
- MAE: {metrics['mae']:.4f}
**Success Criteria:**
- Target RMSE: {target_rmse} → {'✅ ACHIEVED' if
metrics['rmse'] < target_rmse else '❌ NOT
ACHIEVED'}
"""

```

Output: test_metrics + validation_report + visualizations.

Page 9: DEPLOY — Model Deployment

Purpose: Put model into production.

UI Components:

- Deployment Type: REST API, Batch, Stream
- Endpoint Configuration: API path, authentication
- Model Versioning: Auto-increment
- Deploy Button: One-click deployment

Backend Logic:

```

Python
# Save model with version
model_path = f"s3://models/{workflow_id}/v{version}/model.pkl"
s3.upload_file(local_model_path, 'models',
f"{workflow_id}/v{version}/model.pkl")
# Register endpoint
endpoint_url = f"/api/predict/{workflow_id}/v{version}"
# Deploy FastAPI endpoint dynamically

```

Output: deployment_url + model_version + deployment_status.

Page 10: MONITOR — Performance Monitoring

Purpose: Watch performance over time and trigger retraining.

UI Components:

- Performance Dashboard: Metrics over time
- Data Drift Detection: Population Stability Index (PSI)
- Retrain Trigger: Manual or automatic threshold-based

Backend Logic:

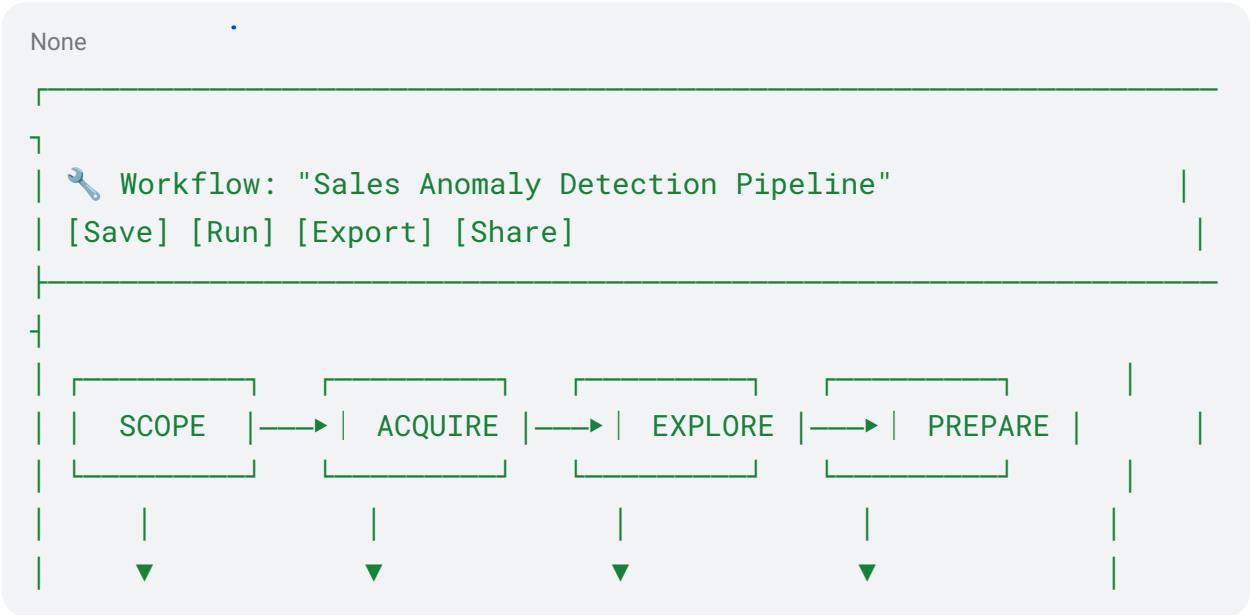
```
Python
def check_drift(reference_data, current_data):
    psi_scores = {}
    for col in reference_data.columns:
        # Calculate PSI for each feature
        psi = calculate_psi(reference_data[col], current_data[col])
        psi_scores[col] = psi
        if psi > 0.2: # Significant drift
            trigger_retrain(col)
    return psi_scores
```

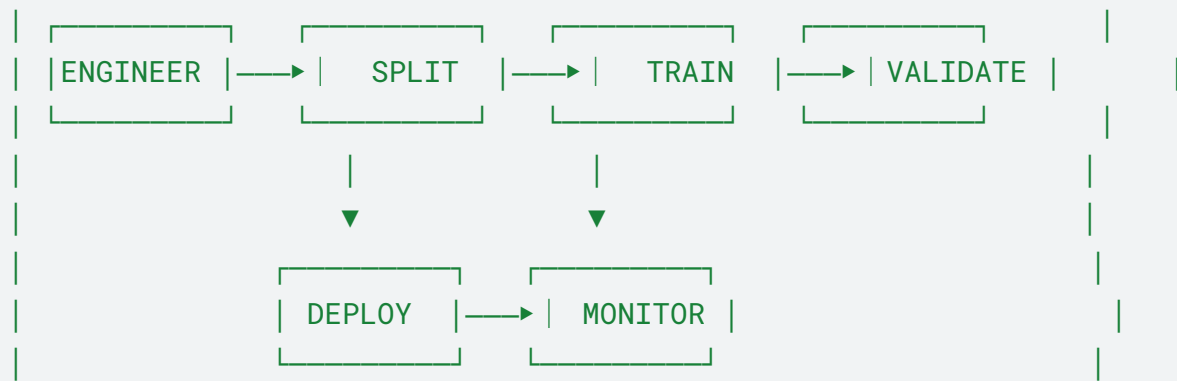
Output: monitoring_dashboard + drift_alerts + retrain_suggestions.

4. The Orchestration Page (Workflow Canvas)

This is the heart of your app — where users connect modules into pipelines.

UI Design (ReactFlow-based)





Node Palette (Drag to canvas):

[SCOPE] [ACQUIRE] [EXPLORE] [PREPARE] [ENGINEER]
[SPLIT] [TRAIN] [VALIDATE] [DEPLOY] [MONITOR]
[Custom Node...]

Execution Log:

✓ SCOPE completed (0.2s)
✓ ACQUIRE completed (1.4s) - dataset: sales_2024.csv
✓ EXPLORE completed (3.1s) - 5 outliers detected
⌚ PREPARE running...

Backend: Workflow Engine

Python

```
class WorkflowEngine:
    def __init__(self, workflow_id):
        self.workflow = self.load_workflow(workflow_id)
        self.nodes = self.workflow['nodes']
        self.edges = self.workflow['edges']
        self.execution_state = {}
```

```

def execute(self):
    # Topological sort to determine execution order
    sorted_nodes = self.topological_sort()
    for node in sorted_nodes:
        # Get inputs from previous nodes
        inputs = self.get_node_inputs(node)
        # Execute the node's function
        result = self.execute_node(node, inputs)
        # Store result for downstream nodes
        self.execution_state[node['id']] = result
        # Update progress
        self.update_progress(node)
    return self.execution_state

def execute_node(self, node, inputs):
    node_type = node['type']
    params = node['params']
    # Route to appropriate handler
    if node_type == 'SCOPE':
        return self.handle_scope(params)
    elif node_type == 'ACQUIRE':
        return self.handle_acquire(params, inputs)
    elif node_type == 'EXPLORE':
        return self.handle_explore(params, inputs)
    # ... continue for all node types

```

Workflow JSON Schema

```

JSON
{
  "workflow_id": "wf_123",
  "name": "Anomaly Detection Pipeline",
  "nodes": [
    {

```

```

    "id": "node_1",
    "type": "SCOPE",
    "position": {"x": 100, "y": 100},
    "params": {
        "problem_type": "anomaly",
        "success_metric": "precision > 0.90"
    }
},
{
    "id": "node_2",
    "type": "ACQUIRE",
    "position": {"x": 300, "y": 100},
    "params": {
        "bucket": "my-data-bucket",
        "file": "transactions_2024.csv"
    }
}
],
"edges": [
    {"source": "node_1", "target": "node_2", "type": "dataflow"}
]
}

```

5. Two Modes of Operation

Mode A: Individual Stage Testing

User can click on any node and run it in isolation with mock data or previous outputs.

Python

Test a single node

@app.post("/workflow/{workflow_id}/test/{node_id}")

def test_node(workflow_id: str, node_id: str, test_input: dict):

node = get_node(workflow_id, node_id)

result = execute_node(node, test_input)


```
    return {"node_id": node_id, "result": result, "status":  
           "tested"}
```

Mode B: One-Go Full Pipeline

User clicks "Run All" and the entire workflow executes end-to-end.

```
Python  
@app.post("/workflow/{workflow_id}/run")  
def run_workflow(workflow_id: str):  
    engine = WorkflowEngine(workflow_id)  
    results = engine.execute()  
    return {"workflow_id": workflow_id, "results": results,  
           "status": "completed"}
```

6. Implementation Roadmap

Phase 1: Foundation (Week 1-2)

1. Set up FastAPI backend with SQLAlchemy
2. Set up React + ReactFlow frontend
3. Implement JWT authentication
4. S3 integration (boto3) for dataset storage

Phase 2: Core Nodes (Week 3-4)

5. Implement SCOPE, ACQUIRE, EXPLORE nodes
6. EDA with pandas-profiling or sweetviz
7. Data visualization with matplotlib/plotly

Phase 3: ML Nodes (Week 5-6)

8. Implement PREPARE, ENGINEER, SPLIT nodes
9. Implement TRAIN with sklearn integration
10. Implement VALIDATE with metrics dashboard

Phase 4: Orchestration (Week 7-8)

11. Workflow engine (topological sort execution)
12. Drag-drop canvas with ReactFlow
13. Node connection validation
14. Execution logging and progress tracking

Phase 5: Deployment & Monitoring (Week 9-10)

15. Model deployment as FastAPI endpoint
16. Monitoring dashboard with drift detection

17. Retraining triggers

7. Key Technical Considerations

For Regression Focus

- Node Extensions: Add ARIMA, Prophet for time-series regression
- Metrics: RMSE, MAE, MAPE, R^2
- Visualizations: Residual plots, Q-Q plots

For Anomaly Detection Focus

- Node Extensions: Isolation Forest, LOF, Autoencoder
- Metrics: Precision, Recall, F1, ROC-AUC
- Visualizations: Anomaly score distribution, t-SNE plots

Scalability

- Use Celery for long-running training jobs
- Use Redis for caching intermediate results
- Store all artifacts (models, cleaned data, plots) in S3

Security

- JWT-based authentication for all APIs
- Encrypted credentials storage
- Role-based access control (admin/user/viewer)

8. Sample Workflow: Anomaly Detection

None

SCOPE: "Detect fraudulent transactions in payment data"

↓

ACQUIRE: "transactions_2024.csv" from S3

↓

EXPLORE: Found 12% missing values, 5% outliers in amount column

↓

PREPARE: Filled missing with median, capped outliers at 99th percentile

↓

ENGINEER: Created features: transaction_frequency, avg_amount_7d

↓

SPLIT: 70% train, 15% val, 15% test (stratified by fraud_label)

↓

TRAIN: Isolation Forest with contamination=0.05

↓

VALIDATE: Precision=0.92, Recall=0.85, F1=0.88 →  SUCCESS

↓

DEPLOY: REST API at /api/predict/fraud/v1

↓

MONITOR: PSI < 0.1 after 30 days →  NO DRIFT

This architecture gives you complete control over your ML pipelines while maintaining the visual, modular experience of n8n. Each stage is a standalone, testable module that can be rearranged to build different algorithms—from simple regression to complex anomaly detection systems.